Explicit Positional Encodings Turn Length Generalisation into Lookup

Anonymous submission

Abstract

Explicit positional encodings are widely held to spoil length generalisation because they place the model at absolute positions it never saw in training. We argue that this diagnosis is wrong, and offer a simpler one: an explicit code lets the model adopt a *length-indexed lookup* that is exact at the training length and undefined everywhere else. The distinction is operational—if the failure is an out-of-distribution position, a better-behaved absolute code can fix it; if it is a table, it cannot, because the table's extent is set by the training length.

The two accounts are not separable in the usual protocol: a sequence of L tokens occupies $2L+1$ positions, so "position beyond the trained range" and "length beyond the trained length" become true at the same moment. We separate them by fixing the total token count at 41 and padding every example, so all 41 positions occur during training and only the effective length varies. With no unseen position left to blame, learned and sinusoidal codes still collapse from 1.00 at L=8 to 0.00 at L=10, while NoPE holds 0.79 and ALiBi 0.87 under the identical protocol.

In a controlled 5×5 study where every cell first reaches 0.998 in-distribution accuracy, we also reject the most natural task-side explanation— that the effect is organised by whether a task's positional rule depends on the sequence length—while confirming that an encoding×task interaction is strong ($\eta_p^2 = 0.95$). One exception constrains the mechanism further: ALiBi, the only scheme that adds position to attention logits rather than to the token representation, is also the only one that partially survives. Explicit position is not harmful in itself; harmful is putting it where it can be memorised.

1 Introduction

A Transformer trained on sequences of length eight will fail on sequences of length sixteen, even when the task is copying. The usual fix is to change the positional encoding (PE), a decision made before pretraining and expensive to reverse, so the diagnosis behind the current advice is worth getting right.

That advice rests on a specific diagnosis: an explicit code assigns the model absolute positions, a longer sequence puts it at positions it never saw, and an embedding trained on positions 0..7 is out of distribution at position 12. Two research programmes follow. One removes the code entirely, on the evidence that a no-PE decoder extrapolates better than five alternatives across ten tasks [6]. The other keeps the code but tries to make it extrapolable—log-decaying biases [2], resolution-maximising relative codes [16], randomised position sampling [13], priming with long examples [5]. Both are treatments for out-of-distribution positions.

We propose that the diagnosis is wrong, and that the phenomenon is simpler. An explicit code does not merely place the model outside its training range; it lets the model adopt a solution that is *indexed by sequence length*, exact at L_{tr} and undefined elsewhere. The difference is operational: if the failure is out-of-distribution positions, a better-behaved absolute code can fix it; if the failure is a lookup table, it cannot, because the table's extent is set by the training length, not by the code's numerical properties.

The two hypotheses are not separable in the standard format, which is why the question has stayed open. A sequence of L tokens occupies $2L+1$ positions, so "position beyond the trained range" and "length beyond the trained length" become true at exactly the same moment. Training at one length and testing longer confounds them by construction, and a probe that merely repeats this at a second training length confounds them again.

We separate them by fixing the total number of tokens. Every example is padded to 41 tokens with the separator at a fixed index, so all 41 positions occur in every training example—including

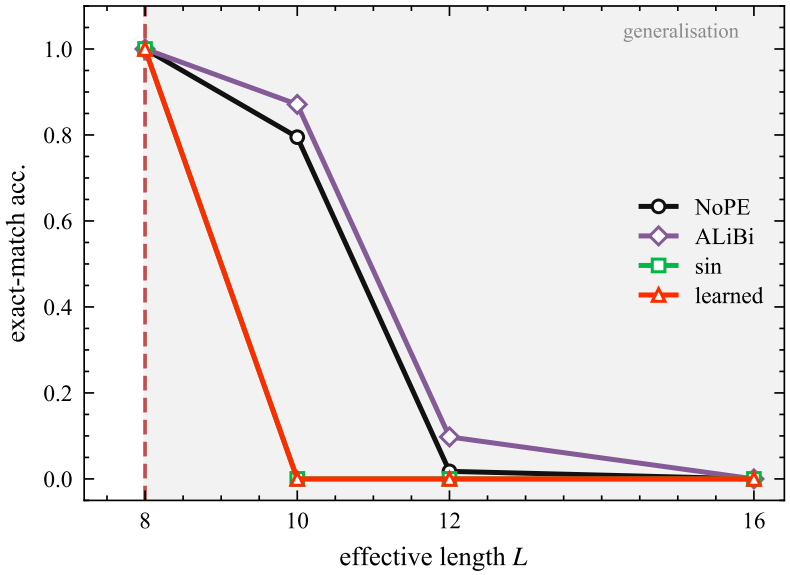


Figure 1: Positions are not the variable. Every training example is padded to 41 tokens with the separator pinned at a fixed index, so all 41 positions occur during training and only the effective length varies. The dashed red line marks the training length; everything to its right is measured at a length never trained on. With every position seen, learned and sinusoidal codes still fall from 1.00 to 0.00 between L=8 and L=10, while NoPE holds 0.79 and ALiBi 0.87. Copy task, one run per condition.

the positions a longer test sequence will use—and only the number of *effective* tokens varies. If the failure is out-of-distribution position, it should disappear. It does not (Figure 1).

With every position seen, learned and sinusoidal codes fall from 1.00 to 0.00 between L=8 and L=10, while NoPE holds 0.79 and ALiBi 0.87 under the same protocol. Position range is therefore not what runs out. The collapse is indexed by length.

Two further results keep the account honest. First, we tested the most natural task-side alternative—that the effect is organised by whether a task's positional rule is written in terms of L—and it fails: across five tasks the ranking of encodings does not reverse between classes (Section 5). The axis is not a property of the input–output relation. Second, one scheme behaves differently: ALiBi retains 0.91 at L=10 on copy where the other three reach zero. ALiBi is the only encoding here that adds position to the attention logits rather than to the token representation—a place where a length-indexed table cannot be written. That is the positive half of the diagnosis, and it is what makes it actionable.

What this changes. If explicit position is harmful because it can be memorised, then the design question is not which code is most extrapolable but where position is allowed to enter. Our data do not settle that question—one exception is a lead, not a result—but they do remove the reason to keep searching for a better absolute code, and they explain why the search has repeatedly come up empty.

Contributions.

1. A diagnosis of why explicit positional encodings harm length generalisation: they admit a length-indexed lookup, not merely an out-of-distribution position (Section 5).

2. A discriminating test between the two accounts: a padded protocol that fixes the total token count so that every position occurs in training and only the effective length varies, plus a controlled $5 \times 5 \times 2$ study in which every cell first reaches 0.998 in-distribution accuracy (Section 4).

3. The rejection of a task-side explanation for the same effect, and one exception (ALiBi) that locates the mechanism at the injection site (Section 6).

2 Related Work

Length generalisation. Transformers fit short sequences but degrade sharply when asked to solve the same task at greater length [1, 3]. The failure is not merely a matter of scale: it persists in large language models and is only partially mitigated by chain-of-thought or scratchpad supervision [1]. Two explanations have been offered. The first is architectural: self-attention with a fixed number of layers and heads cannot represent certain length-dependent computations, such as periodic finite-state languages, unless depth or width grows with the input [4]. The second is algorithmic: a task extrapolates when it admits a short program over the attention primitives that works at every length [19, 21], and shallow Transformers accordingly learn "shortcut" solutions to automata that fit the training lengths and break outside them [7]. Both explanations locate the obstacle in the computational structure of the task rather than in the position code—which is the premise our experiments end up supporting better than the position-code hypothesis we set out to test.

Positional encodings. Absolute encodings, whether sinusoidal [17] or learned, give each position a distinct vector. Relative schemes instead inject pairwise information: learned relative representations [14], T5's bucketed bias [12], rotary embeddings that rotate queries and keys by a position-dependent angle [15], and ALiBi's linear distance penalty [11]. The last two were introduced precisely as length-extrapolation devices, yet head-to-head comparisons on downstream reasoning tasks have been unflattering to them [6].

Designs built for extrapolation. A parallel line of work engineers encodings for the extrapolation regime directly: log-decaying attention biases derived from receptive-field analysis [2], relative encodings that maximise attention resolution [16], and randomised position sampling during training so that a model never sees a narrower range of positions than it will meet at test time [13]. Arithmetic has served as a stress test: relative embeddings suffice for addition but not multiplication, where priming the training set with a handful of long examples is required [5]. A recent survey organises these methods by encoding family and by the mechanism each attributes the failure to [20].

The no-PE result and its scope. Kazemnejad et al. [6] evaluate five schemes across ten tasks and conclude that having no positional encoding at all is better than any explicit alternative; they further show that a no-PE decoder can represent absolute and relative encodings in principle, while in practice resembling a relative scheme. Follow-up work probes the limits of that result, finding that no-PE models still extrapolate only to a bounded context and tracing the breakdown to attention becoming unfocused [18]. Our paper takes the same result as its starting point and asks a question the aggregate ranking cannot answer: whether the advantage of NoPE holds uniformly, or whether it too is conditional on the task.

Mechanistic and training-dynamics evidence. Small algorithmic models are tractable enough to reverse-engineer, and much of what is known about length-dependent behaviour comes from that setting: grokking was characterised as a transition from memorisation to a structured circuit rather than a change in fit [8, 10], and induction heads were identified as a mechanism for copying from context [9]. We do not offer a mechanistic account of our own result—Section 7 names this as the main gap—but we frame the question so that such an account would be informative, and we report behavioural evidence that constrains it.

This paper. The no-PE result is stated as a ranking aggregated over a task battery. We do not dispute the individual measurements; we dispute that they compose into a task-independent recommendation. Our contribution is to supply the missing conditioning variable and to test whether the ranking it induces is constant across tasks. In this respect we follow the broader lesson of the small-scale algorithmic literature, where apparently uniform phenomena turn out to be task- and data-dependent [8, 10].

Table 1: The five tasks, ordered by length-coupling. $S_L(i)$ is the input positions read by output i; "rule" is the corresponding positional rule $\rho(i,L)$. Only the two L-coupled tasks have a rule that cannot be evaluated without knowing the test length.

Task	$S_L(i)$	Rule $\rho(i,L)$	Class
copy	$\{i\}$	i	decoupled
prefix-XOR	$\{1,\ldots,i\}$	$1..i$	decoupled
sort	—	none	position-free
swap-halves	$\{(i+L/2) \bmod L\}$	$(i+L/2) \bmod L$	**coupled**
reverse	$\{L+1-i\}$	$L+1-i$	**coupled**

3 Length-Coupling: A Task Axis for Positional Encoding

Our starting point is a simple observation about the studies that motivate this paper. When the effect of a design choice is summarised as a single ranking over a task battery, the ranking is only interpretable if that effect is roughly homogeneous across tasks. We argue that for positional encoding it is not, and that the relevant axis of heterogeneity is a property of the *task*, not of the model.

3.1 Positional rules

Let a task family be a collection of maps indexed by length,

$$\mathscr{T} = \{T_L\}_{L\geq 1}, \qquad T_L : \mathscr{X}^L \longrightarrow \mathscr{Y}^L, \tag{1}$$

and write $T_L(x)_i$ for the i-th output token of $T_L(x)$. For a given input $x \in \mathscr{X}^L$, let $S_L(i;x) \subseteq [L]$ be the minimal set of input positions that $T_L(x)_i$ depends on, i.e. the support of $T_L(x)_i$ when it is regarded as a function of the input positions with the values held fixed.

Definition 1 (Positional rule)**.** $\mathscr{T}$ admits a positional rule if there is a function $\rho : [L] \times \mathbb{N} \to 2^{[L]}$ such that for every L, every $i \in [L]$ and every $x \in \mathscr{X}^L$,

$$S_L(i;x) = \rho(i,L). \tag{2}$$

That is, the input positions an output token reads are fixed by the output index and the sequence length alone, independently of the values in x.

Definition 2 (Length-coupling)**.** A task family with positional rule ρ is *length-coupled* if ρ depends on its second argument,

$$\exists\, i, L, L' : \quad \rho(i,L) \neq \rho(i,L'), \tag{3}$$

and *length-decoupled* if it does not, i.e. if $\rho(i,L) = \rho_0(i)$ for all L and some ρ_0. A family that admits no positional rule in the sense of Section 3 is *position-free*.

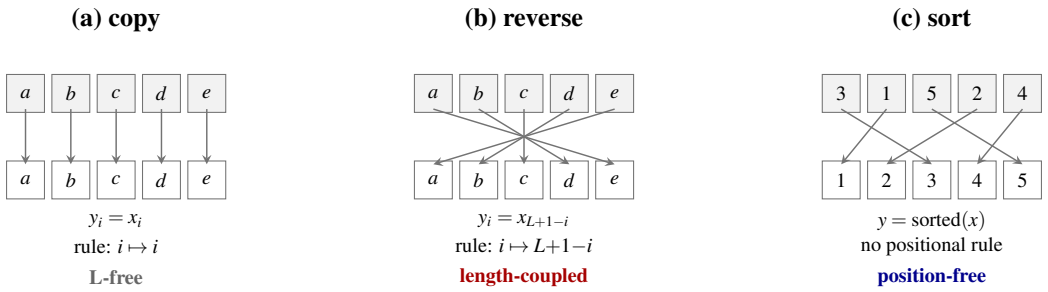
Table 1 instantiates the taxonomy on the five tasks we study. *Copy* reads input i to produce output i, and *prefix-XOR* reads the prefix $1..i$; in both cases the rule is the same for every L. *Reverse* maps output i to input $L+1-i$, and *swap-halves* maps it to $(i+L/2) \bmod L$; both rules are stated in terms of L and therefore change as L changes. *Sort* is position-free: which input position supplies output i is decided by the relative magnitudes of the values, so no positional rule exists—yet its outputs still depend on the entire input, which is why a coarse notion such as "global vs. local" would misclassify it as hard for the same reason as reverse.

3.2 Three places position can enter

Every encoding we compare can be written as a special case of a single causal attention layer,

$$\mathrm{Attn}(Q,K,V)_i = \sum_{j\leq i} \frac{\exp\big(\langle q_i, k_j\rangle/\sqrt{d_h} + B_{ij}\big)}{\sum_{j'\leq i}\exp\big(\langle q_i, k_{j'}\rangle/\sqrt{d_h} + B_{ij'}\big)}\, v_j, \tag{4}$$

with $q_i = W_Q h_i$, $k_j = W_K h_j$, $v_j = W_V h_j$ and h_i the residual stream at position i. Position can be injected at exactly three sites, and the encodings we study occupy all three:



Prediction: an explicit code should help in (b) only.
Observed here: NoPE is best in (a), (b) *and* (c) — the prediction fails.

Figure 2: The axis we test. A task is *length-coupled* when the rule mapping an output position to the input positions it reads is written in terms of the sequence length L; (b) is coupled because the rule contains L, while (a) uses the same rule at every L. Panel (c) shows why a coarse "local vs. global" distinction would fail: sorting reads the whole input, yet has no positional rule at all, since which input supplies output i is decided by the values. Our hypothesis was that an explicit position code should matter only in (b); the experiments in Section 5 contradict it.

(i) into the token representation, $h_i = e(x_i) + p_i$, with p_i either learned or a fixed sinusoid (APE, sin);

(ii) into the query/key geometry, $q_i \mapsto R_{\Theta,i} q_i$, so that $\langle q_i, k_j \rangle$ depends on $i - j$ alone (RoPE);

(iii) into the logits through a bias B_{ij} that carries no information about the token content (ALiBi);

and NoPE is the fourth case, in which all three sites are left untouched and order is conveyed only by the causal mask $j \leq i$. Writing the encodings this way matters for the interpretation of our results: our design varies site (i) against site (iii) and against nothing, but never holds the positional signal fixed while varying only its injection site. We return to this in Section 6.

3.3 Why coupling should change the sign of the PE effect

Consider a decoder-only model trained at a single length L_{tr} and evaluated at $L > L_{\text{tr}}$.

Decoupled tasks. The positional rule is evaluable at any length: to produce output i the model must read input i (or $1..i$). In a causal Transformer the ordering of tokens is already available through the attention mask—the i-th position can only attend to positions $\leq i$—so an explicit position code supplies information that the architecture already encodes. Worse, for absolute codes the embeddings at positions $> L_{\text{tr}}$ are either untrained (learned APE) or outside the fitted range (sinusoidal), so at test time the code is a distribution shift with no compensating benefit. We therefore expect explicit PEs to be neutral-to-harmful here, reproducing the conclusion of Kazemnejad et al. [6].

Coupled tasks. The rule $L+1-i$ cannot be evaluated without L, so the model must (i) infer the length of the current sequence and (ii) compute a length-dependent index. This is a qualitatively different demand. A relative code (ALiBi, RoPE) supplies only distances, from which L must still be recovered; a no-PE model must recover ordering and length implicitly from attention patterns. An analytic absolute code, by contrast, exposes the index i directly and—because it is a fixed function of position rather than a lookup table—remains defined at unseen positions. If any explicit code helps, it should be this one, and it should help *here* rather than on decoupled tasks.

Position-free tasks. Output order is determined by values, so no positional information is required beyond what the mask provides; PE effects should be small.

This yields two predictions that the aggregate-ranking view cannot express:

P1 The dispersion of accuracy across PEs is larger for coupled than for decoupled tasks.

P2 The ranking of PEs reverses between the two classes: NoPE is competitive on decoupled tasks and relatively worse on coupled tasks, while an analytic absolute code moves in the opposite direction.

Both are statements about an interaction, so they must be tested with a factorial design rather than by averaging over tasks.

4 Experimental Setup

Tasks. All five tasks map a sequence of L symbols from a 10-symbol alphabet to an output sequence of the same length, formatted as $\texttt{x}_{1..\texttt{L}}$ <SEP> $\texttt{y}_{1..\texttt{L}}$ <EOS>. Cross-entropy is applied only to the output segment; accuracy is exact match over all L output tokens. Training uses $L_{\text{tr}} = 8$; evaluation uses $L \in \{8, 10, 12, 16, 20\}$, so L=8 measures in-distribution fit and the rest measure extrapolation.

Positional encodings. All five conditions are instantiations of (4). NoPE sets $p_i = 0$, $B_{ij} = 0$ and applies no rotation, so order is conveyed only by the causal mask. *learned* absolute embeddings use $h_i = e(x_i) + p_i$ with $p_i \in \mathbb{R}^d$ trained, one vector per position. *sin* uses the fixed sinusoid of Vaswani et al. [17],

$$p_{i,2k} = \sin(i\,\omega_k), \qquad p_{i,2k+1} = \cos(i\,\omega_k), \qquad \omega_k = 10000^{-2k/d}. \tag{5}$$

rope rotates queries and keys within each head, $q_i \mapsto R_{\Theta,i}\, q_i$ with $R_{\Theta,i}$ block-diagonal in 2×2 rotations of angle $i\theta_k$ and $\theta_k = 10000^{-2k/d}$, so that $\langle q_i, k_j \rangle$ depends on $i - j$ alone [15]. *alibi* leaves the residual stream untouched and sets

$$B_{ij} = -m_h\,(i - j), \qquad m_h = 2^{-8h/H}, \tag{6}$$

for head $h \in \{1, \ldots, H\}$ [11]. Four of these—NoPE, learned absolute, rotary and ALiBi—appear in the five-way comparison of Kazemnejad et al. [6]. We substitute a fixed sinusoidal code for their T5-style bucketed relative bias [12], so our battery overlaps with theirs rather than reproducing it; we record the substitution explicitly because it limits how directly the numbers can be compared across the two studies.

Model. A decoder-only Transformer with 4 layers, d_{model}=256, 8 heads, pre-layered-norm blocks, GELU MLP with hidden width $4d$, and a tied output head: about 3.2M parameters. All PE variants use the same architecture and differ only in how position enters.

Optimisation. AdamW with learning rate 5×10^{-4}, $\beta = (0.9, 0.98)$, weight decay 0.01, batch size 128, 2000 steps, and a 100-step linear warmup followed by cosine decay. Gradients are clipped to norm 1.0. Each step draws a fresh batch from the task distribution rather than sampling from a fixed finite training set; every PE therefore sees the same number of i.i.d. examples, which removes memorisation of a particular training sample as an explanation for any difference we observe.

Evaluation and the "solved" control. For each length we evaluate 512 fresh sequences with a fixed evaluation seed, so all conditions are scored on identical inputs. A condition is counted as *solved* when in-distribution accuracy at L=8 reaches 0.95. This control matters: extrapolation failure is only interpretable for models that learned the task in the first place, and without it a PE can appear to "fail to extrapolate" when it in fact failed to fit. We report unsolved conditions explicitly rather than dropping them silently.

Seeds and aggregation. Every cell of the 5×5 design is run with 2 random seeds. Our primary summary statistic is the *extrapolation score*,

$$\text{ES}(\pi) = \frac{1}{|\mathscr{L}_{\text{te}}|} \sum_{L \in \mathscr{L}_{\text{te}}} \text{Acc}_L(\pi), \qquad \mathscr{L}_{\text{te}} = \{10, 12, 16, 20\}, \tag{7}$$

Table 2: Extrapolation score ES (mean exact-match accuracy over $L \in \{10, 12, 16, 20\}$), averaged over seeds and restricted to solved runs. Best per row in bold. Two length-coupled tasks are marked †. Every cell using an explicit additive or rotary code sits at 0.00; only ALiBi on copy departs from the pattern.

Task	NoPE	learned	sin	RoPE	ALiBi
copy	**0.34**	0.00	0.00	0.00	0.31
prefix-XOR	**0.00**	0.00	0.00	0.00	0.00
sort	**0.76**	0.00	0.00	0.00	0.00
swap-halves†	0.00	0.00	0.00	0.00	0.00
reverse†	**0.29**	0.00	0.00	0.00	0.00

where $\mathrm{Acc}_L(\pi)$ is the exact-match rate of encoding π at test length L, averaged over seeds and restricted to solved runs. We report the spread across seeds alongside each mean, and we test the task×PE interaction with the two-way model

$$\mathrm{ES}_{ijk} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \varepsilon_{ijk}, \tag{8}$$

with task i, encoding j and seed k treated as a replicate. The residual degrees of freedom (25) are small and we interpret the test accordingly.

5 Results

All 25 cells are solved by our criterion (in-distribution accuracy ≥ 0.95; the minimum observed is 0.998), so every comparison below is between models that learned the task.

The cliff is not specific to one task. Figure 3 repeats the pattern of Figure 1 across the battery. With a learned, sinusoidal or rotary code, accuracy is 1.00 at the training length and 0.00 at the next one tested, on every task where the task can be learned at all. NoPE instead decays smoothly where the task permits extrapolation (copy 0.96 at L=10, reverse 0.87, sort 1.00). Twenty-one of the twenty-five cells have ES below 0.01, eighteen exactly zero; the discrimination between methods comes from the four cells that clear it.

The failure is not an out-of-distribution position. The standard format cannot distinguish the two accounts, since $2L+1$ positions grow with L. Figure 1 removes the confound: every example is padded to a fixed 41 tokens with the separator pinned at index 21, so all positions occur in training and only the effective length varies. Under this protocol the learned and sinusoidal codes still fall from 1.00 at L=8 to 0.00 at L=10, whereas NoPE retains 0.79 and ALiBi 0.87, decaying to 0.02 and 0.10 at L=12. Padding costs some accuracy overall—NoPE reaches 0.96 at L=10 in the standard format and 0.79 here, plausibly because half the sequence is now filler—but the ordering is unchanged and the collapse is undiminished. With no unseen position left to blame, the failure has to be indexed by length.

A task-side explanation does not fit. Before settling on the lookup account we tested the most natural alternative: that the effect is organised by whether a task's positional rule is written in terms of L (Section 3). Under that account the ranking of encodings should reverse between classes, with NoPE losing its advantage on the coupled tasks. It does not. NoPE is at or near the top of every task, and the spread between the best and worst encoding is 0.34 (copy), 0.76 (sort) and 0.29 (reverse)—larger on the decoupled side (0.37 mean) than the coupled side (0.15). What the data do show is that heterogeneity is real: a two-way ANOVA treating seed as a replicate (n=50) gives a task×encoding interaction of $F(16, 25) = 32.3$, $p < 10^{-3}$, $\eta_p^2 = 0.95$. Encodings do behave differently across tasks; the difference is just not organised by coupling. We report this because it cuts against the explanation we set out to test, and because a null interaction would have been a weaker finding. The test treats seeds as independent replicates, an approximation that if anything understates the residual variance.

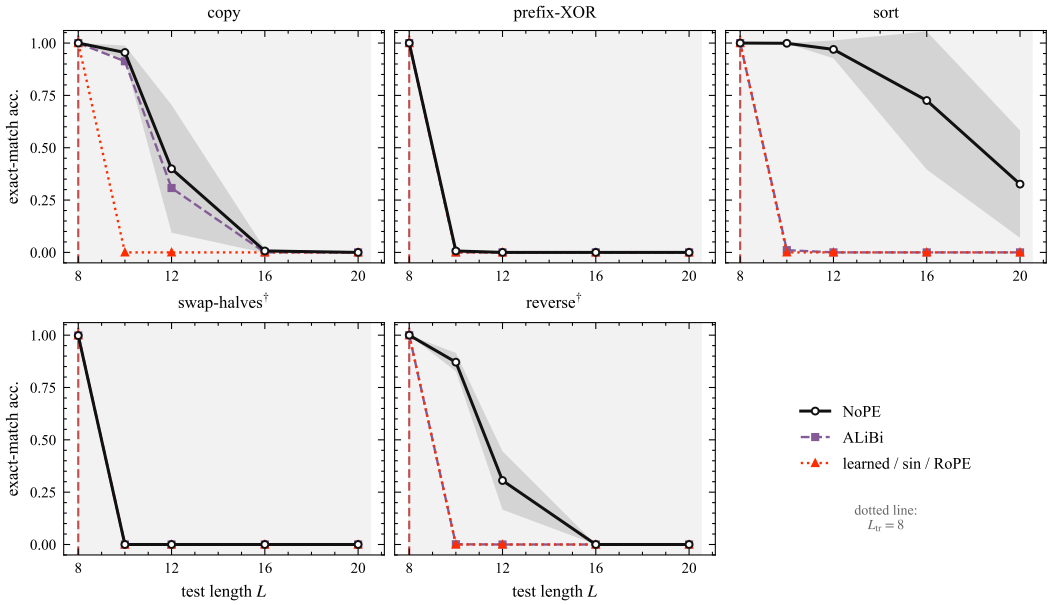


Figure 3: Exact-match accuracy versus test length, one panel per task, mean over seeds with ± 1 s.d. The dotted line marks the training length. Learned, sinusoidal and rotary codes coincide numerically, so they are drawn as one curve; all three fall to zero at the first length beyond training. NoPE instead decays gradually on the three tasks it can extrapolate at all, and fails on the remaining two. Tasks are ordered by length-coupling, coupled ones marked $*$.

Where every method fails, the obstacle is not positional. On prefix-XOR and swap-halves, *every* condition including NoPE scores at or below 0.01 despite reaching ≥ 0.998 in-distribution. The obstacle there is the difficulty of representing the underlying algorithm in a length-invariant way, the phenomenon studied by Zhou et al. [21], and no choice of positional encoding can address it. Keeping these tasks in the battery is what makes that visible: it bounds what any PE intervention could achieve.

6 Discussion

6.1 Why the failure is a cliff

A model that had learned "copy the token at the same index" would degrade gradually: the rule still holds at L=10. We observe no such degradation. With a learned, sinusoidal or rotary code, accuracy is 1.00 at the training length and 0.00 at the next one tested (Figure 3). A cliff of this shape means the model did not learn the rule but something that replicates it at L_{tr} and is undefined elsewhere—a length-indexed table.

The obvious objection is that this is just the usual out-of-distribution story: positions beyond $2L_{\text{tr}}$ were never seen. It is not, and the padded experiment is what shows it. Once every position occurs during training, a model relying on position would be fine; the learned and sinusoidal models are not ($1.00 \rightarrow 0.00$ at L=10, Figure 1). What runs out is not the position range but the length range, which is exactly what a table indexed by length would do.

6.2 The one exception, and what it points at

ALiBi is the only scheme that survives partially—0.91 at L=10 on copy in the standard format, 0.87 under padding, against 0.00 for the other three. It is also the only scheme that adds position to the attention logits instead of to the token representation. This matters for the diagnosis rather than being a footnote: a bias on logits can be ignored by a head that does not need it, whereas a perturbation of the representation has to be compensated for. A length-indexed table can be written into a representation; it is much harder to write one into a logit offset that a head may simply not use.

We did not vary the injection site while holding the positional signal fixed, so this remains a hypothesis rather than a result. It is, however, a falsifiable one with an obvious next experiment, and it changes what the field should try next.

6.3 A task-side explanation we ruled out

Before settling on the lookup account we tested the most natural alternative from the task side: that the effect is organised by whether a task's positional rule is written in terms of L (Section 3). Under that account the ranking of encodings should reverse between classes, with NoPE losing its advantage on the coupled tasks. It does not. The dispersion across encodings is if anything larger on the decoupled side (0.37) than the coupled side (0.15). What is real is heterogeneity: the task×encoding interaction reaches $\eta_p^2 = 0.95$. The encodings do behave differently across tasks—just not along this axis. We report the failure because a null interaction would have been a weaker finding, and because we set out to test this explanation rather than to confirm it.

6.4 Two mundane explanations we ruled out

Insufficient fitting. Every cell reaches at least 0.998 in-distribution accuracy before it is scored, and training loss is at or below 10^{-3}. The models fit; the failures are extrapolation failures.

Drift of the structural marker. The <SEP> token sits at absolute position L in the standard format, so it moves as L changes. Pinning it to a fixed index does not rescue the sinusoidal model. This probe is subsumed by Figure 1, which fixes the separator and shows the collapse unchanged.

6.5 What this means for practice

If an explicit code is harmful because its effect can be memorised, then searching for a more extrapolable absolute code is aimed at the wrong failure: the code's numerical behaviour at large positions is irrelevant if the solution learned is a table whose extent is set by the training length. The two interventions that follow are to omit the code, or to put it somewhere a table cannot be written—as ALiBi does. Neither is established at the scale of pretrained language models, and our evidence for the second is one task and one run per condition.

6.6 Where this leaves the aggregate-ranking critique

The premise survives and the remedy does not. A ranking aggregated over tasks does hide a strong interaction ($\eta_p^2 = 0.95$), so the methodological worry is real. But the conditioning variable we proposed carries no predictive power, and the account that does fit—length-indexed lookup—is not a property of the task at all. The axis we were looking for is probably not on the task side.

7 Limitations

Scale. Our models have 3.2M parameters and are trained from scratch on synthetic sequence tasks. Nothing in our argument requires scale, but nothing in our experiments establishes that the same conditioning holds for pretrained language models, where positional codes interact with token distributions and with training that already spans many lengths. We regard the present study as a controlled test of a mechanism, not as a claim about frontier models.

One training length. All runs use L_{tr}=8. A stronger version of our claim would show that the coupling classification predicts the *shape* of the extrapolation curve across several training lengths, not just the sign of a difference at one. This is the most important extension we have not run.

Online sampling. Each step draws a fresh batch rather than sampling from a fixed finite training set. This removes memorisation as an explanation and equalises the number of examples across conditions, but it also makes the task easier than the fixed-train-set protocol used in some prior work, so absolute accuracies are not directly comparable across papers.

Coverage of the taxonomy. Five tasks is a small battery, and the position-free class is represented by a single task (sort). The two coupled tasks, while both length-dependent, differ in how the

dependence enters (complement versus rotation), so we cannot separate "coupled" from "coupled in this particular way".

Confound in the learned-APE condition. At test lengths the learned embedding table is queried at positions never updated during training. This is faithful to how learned absolute codes behave in deployment, but it means the learned condition conflates "absolute position is unhelpful" with "the untrained tail of a lookup table is unhelpful". A variant with a length-extrapolable initialisation would separate the two; we did not run one.

Statistical power. Two seeds per cell leave only 25 residual degrees of freedom for the interaction test, and treating seeds as replicates is an approximation. The interaction we report is far from marginal ($F(16,25) = 32.3$), but the cell-level means it is computed from are noisy enough that we would not defend a claim about any single pairwise difference. A five-seed replication is the obvious next step and we have not run it.

Behavioural evidence only. We report accuracy, not mechanism. Claims about *why* a coupled task is harder—whether models recover length from attention statistics, whether they implement a position-arithmetic circuit—would require probing or circuit analysis that we have not performed.

8 Conclusion

The question we began with was not which positional encoding is best, but what property of a task predicts whether a positional encoding will help. We proposed one such property—whether the task's positional rule is written in terms of the sequence length—and tested it with a factorial design that separates "failed to fit" from "failed to extrapolate". The answer was no: the coupling class did not predict the sign of the effect, and NoPE was never worse than an explicit code, including on the two tasks constructed to favour its competitors. What we can say with more confidence than before is narrower but firmer: the advantage of omitting positional encoding survives the condition we designed to break it.

The broader methodological point is that recommendations about architectural components are conditional objects. An aggregate ranking over a task battery is a summary of a distribution, and if that distribution is heterogeneous the summary can invert the sign of the effect for any given task. Reporting the conditioning variable alongside the ranking costs little and makes the recommendation actionable: a practitioner who knows their task's coupling class can choose a positional encoding deliberately rather than by reputation.

References

[1] Cem Anil, Yuhuai Wu, Anders Andreassen, Aitor Lewkowycz, Vedant Misra, Vinay Ramasesh, Ambrose Slone, Guy Gur-Ari, Ethan Dyer, and Behnam Neyshabur. Exploring length generalization in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 38546–38556, 2022.

[2] Ta-Chung Chi, Ting-Han Fan, Alexander Rudnicky, and Peter Ramadge. Dissecting transformer length extrapolation via the lens of receptive field analysis. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13522–13537, 2023.

[3] Grégoire Delétang, Anian Ruoss, Jordi Grau-Moya, Tim Genewein, Li Kevin Wenliang, Elliot Catt, Chris Cundy, Marcus Hutter, Shane Legg, Joel Veness, and Pedro A. Ortega. Neural networks and the chomsky hierarchy. In *International Conference on Learning Representations (ICLR)*, 2023.

[4] Michael Hahn. Theoretical limitations of self-attention in neural sequence models. *Transactions of the Association for Computational Linguistics*, 8:156–171, 2020.

[5] Samy Jelassi, Stéphane d'Ascoli, Carles Domingo-Enrich, Yuhuai Wu, Yuanzhi Li, and François Charton. Length generalization in arithmetic transformers. *arXiv preprint arXiv:2306.15400*, 2023.

[6] Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. The impact of positional encoding on length generalization in transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[7] Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Transformers learn shortcuts to automata. In *International Conference on Learning Representations (ICLR)*, 2023.

[8] Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations (ICLR)*, 2023.

[9] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context learning and induction heads. Transformer Circuits Thread, 2022. arXiv:2209.11895.

[10] Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.

[11] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations (ICLR)*, 2022.

[12] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), 2020.

[13] Anian Ruoss, Grégoire Delétang, Tim Genewein, Jordi Grau-Moya, Róbert Csordás, Mehdi Bennani, Shane Legg, and Joel Veness. Randomized positional encodings boost length generalization of transformers. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 1889–1903, 2023.

[14] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *Proceedings of NAACL-HLT*, pages 752–761, 2018.

[15] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[16] Yutao Sun, Li Dong, Barun Patra, Shuming Ma, Shaohan Huang, Alon Benhaim, Vishrav Chaudhary, Xia Song, and Furu Wei. A length-extrapolatable transformer. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14590–14604, 2023.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.

[18] Jie Wang, Tao Ji, Yuanbin Wu, Hang Yan, Tao Gui, Qi Zhang, Xuanjing Huang, and Xiaoling Wang. Length generalization of causal transformers without position encoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 14024–14040, 2024.

[19] Gail Weiss, Yoav Goldberg, and Eran Yahav. Thinking like transformers. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139, pages 11080–11090, 2021.

[20] Liang Zhao, Xiachong Feng, Xiaocheng Feng, Weihong Zhong, Dongliang Xu, Qing Yang, Hongtao Liu, Bing Qin, and Ting Liu. Length extrapolation of transformers: A survey from the perspective of positional encoding. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 9959–9977, 2024.

[21] Hattie Zhou, Arwen Bradley, Etai Littwin, Noam Razin, Omid Saremi, Josh Susskind, Samy Bengio, and Preetum Nakkiran. What algorithms can transformers learn? A study in length generalization. In *International Conference on Learning Representations (ICLR)*, 2024.